Intent as a Conserved Quantity: Hamiltonian Monitoring of Agentic and Recursively Self-Improving Systems

Tom Butler · https://atodev.xyz · 13 October 2026

Abstract

Long-running agents and systems that modify themselves can drift from their original intent without any single step being wrong. Monitors that score one step at a time have no memory of the trajectory and cannot say which input caused a departure. We propose a monitoring framework built on Hamiltonian mechanics. The agent's trace is embedded as a trajectory in phase space; the original intent is the minimum of a potential well; and an energy function, the intent Hamiltonian, is compared after every step with a budget calibrated on benign runs. Energy above the budget is the drift signal. The same construction gives an energy ledger that attributes drift to input channels, and a gate that accepts a self-modification only if named behavioural invariants are unchanged from generation zero.

We describe the components, the runtime procedure and a 289-line reference implementation. In a synthetic experiment of 1,200 scripted traces, at a threshold with no false alarms on benign traces, the monitor detected 300 of 300 drifting traces and a memoryless similarity baseline detected 267 of 300. We also report two negative results. The kinetic term of the Hamiltonian added nothing measurable over the potential term, and an energy budget calibrated on two tasks produced many false alarms on one of three held-out tasks. The experiment uses scripted topic-change drift and does not establish performance on real agent workloads; we give a protocol for that evaluation.

1. Introduction and problem statement

An autonomous agent that runs for hundreds of steps, or a system that rewrites its own prompts, tools or code, can end up pursuing something other than what it was asked to do. No single step need be wrong for this to happen. We call the accumulated departure *intent drift*, and this paper proposes a way to measure it continuously.

1.1 Three kinds of drift

- **Within-run goal drift.** Over a long task the agent's working objective shifts by small increments: a sub-task becomes the task, or a convenient proxy replaces the real goal. Goal drift of this kind has been measured in current language-model agents [4].
- **Input-induced drift.** A tool result, retrieved document or message from another agent pushes the agent off course. Prompt injection is the adversarial case [5]; a merely distracting search result is the benign one.
- **Generational drift.** In recursive self-improvement, each generation edits the next. A change that looks like an improvement on the current benchmark may alter behaviour the original intent depended on, and the altered system is then the one judging the next change.

1.2 Why existing monitors fall short

The common monitors are a similarity threshold between the current step and the task description, a second model acting as judge, and static rules on actions. Each evaluates one step at a time, which leaves three gaps.

1. **No memory.** A step-wise check cannot tell a brief, purposeful excursion from the tenth step of a steady slide. Both can sit at the same distance from the goal.
2. **No dynamics.** Direction and speed of travel carry information. An agent moving quickly away from its goal is in a different condition from one resting at the same distance, and a pointwise score treats them alike.
3. **No attribution.** When drift is detected, a pointwise score does not say which input caused it.

1.3 Contribution

We treat the agent's trajectory as motion in a phase space and define an energy function, the *intent Hamiltonian*, whose minimum is the original intent. In a closed mechanical system that energy would be conserved. An agent is not a closed system, so the energy changes, and the change is the quantity we monitor: energy gained is work done on the agent by something other than its intent.

The paper makes four contributions.

1. A formal construction of the phase space and intent Hamiltonian for text-based agent traces (Section 3).
2. Extensions that reuse the same construction, among them per-input energy accounting and an invariant-preservation gate for self-modification (Section 3.7).
3. A component architecture and runtime procedure (Sections 4 and 5), with a reference implementation (Section 6).
4. A controlled synthetic experiment comparing the monitor with a memoryless similarity baseline (Section 7).

The mechanics here is a modelling choice and not a physical claim. Nothing guarantees that agent trajectories obey Hamilton's equations. The framework is useful to the extent that its diagnostics separate drifting runs from healthy ones, which is an empirical question that Section 7 begins to answer and Section 8 qualifies.

2. Background

This section states the pieces of classical mechanics the framework borrows, and what each is used for. Arnold [1] is the standard reference for 2.1, 2.3 and 2.4, and van der Schaft and Jeltsema [2] for 2.2.

2.1 Phase space and the Hamiltonian

A mechanical system is described by generalised positions q and momenta p. The pair (q, p) is a point in phase space, and the Hamiltonian H(q, p) is the total energy, kinetic plus potential. Hamilton's equations give the motion:

$$\dot{q} = \frac{\partial H}{\partial p}, \qquad \dot{p} = -\frac{\partial H}{\partial q}$$

When H has no explicit time dependence and the system is closed, H is constant along every trajectory. A trajectory with energy E is confined to the region where the potential satisfies V(q) ≤ E. The boundary V(q) = E is the set of turning points.

Use here: the potential defines what staying on intent means, the energy budget defines how far exploration may go, and a change in H signals an outside influence.

2.2 Open systems and ports

Real systems exchange energy with their surroundings. Port-Hamiltonian theory writes the energy balance explicitly: the rate of change of H equals the power supplied through the system's ports minus the power dissipated.

$$\frac{dH}{dt} = \sum_j u_j^\top y_j \; - \; D(q,p), \qquad D \geq 0$$

A system is *passive* if it cannot deliver more energy than it has received.

Use here: each tool, data source or peer agent is a port, and the energy balance attributes drift to the port that supplied it.

2.3 Symmetries and conserved quantities

Noether's theorem ties each continuous symmetry of a system to a conserved quantity. A transformation of phase space that preserves the form of Hamilton's equations is called canonical; conserved quantities stay conserved under canonical transformations that respect the symmetry.

Use here: a self-modification is treated as a transformation of the system, and a set of behaviours that must not change plays the role of the conserved quantities. The analogy is structural. We do not claim the modifications are canonical in the technical sense, only that preservation of named invariants is the test applied.

2.4 Phase-space volume

Liouville's theorem states that Hamiltonian flow preserves phase-space volume: a cloud of initial conditions may change shape, but not volume. Lyapunov exponents measure how fast neighbouring trajectories separate.

Use here: for an ensemble of runs started from slightly different prompts, contraction of the cloud indicates collapse onto a fixed behaviour and rapid expansion indicates sensitivity to initial conditions.

3. Formal framework

The framework maps a sequence of text steps to a trajectory in phase space, assigns that trajectory an energy, and raises an alarm when the energy exceeds a budget.

3.1 Trace and embedding

An agent run is a sequence of steps x_0, x_1, ..., each a piece of text: a plan, a tool call with its arguments, a summary of a tool result, or a proposed self-edit. An embedder φ maps each step to a unit vector e_t = φ(x_t) on the sphere in d dimensions. Any sentence embedder can serve [7].

3.2 State and momentum

Single steps are noisy: two consecutive on-task sentences can be far apart in embedding space. The position is therefore a smoothed state, which gives the system inertia.

$$q_0 = e_0, \qquad q_t = \frac{(1-\alpha)\, q_{t-1} + \alpha\, e_t}{\lVert (1-\alpha)\, q_{t-1} + \alpha\, e_t \rVert}, \qquad p_t = m\,(q_t - q_{t-1})$$

The smoothing rate α lies in (0, 1]. With α = 1 the state is the latest step alone and the construction has no memory. The mass m sets how much weight momentum carries in the energy.

3.3 The intent anchor

The intent is given as one or more statements s_1, ..., s_n: the task, and optionally its declared sub-goals. The anchor is their normalised mean embedding.

$$q^{*} = \frac{\sum_i \varphi(s_i)}{\lVert \sum_i \varphi(s_i) \rVert}$$

The anchor is computed once, before the run, and never updated. Section 4 treats its protection as a component requirement.

3.4 The intent Hamiltonian

$$H_t = \underbrace{\frac{\lVert p_t \rVert^2}{2m}}_{T_t} + \underbrace{\frac{k}{2}\, \lVert q_t - q^{*} \rVert^2}_{V_t}$$

Because q_t and q* are unit vectors, the potential reduces to V_t = k (1 − cos θ_t), where θ_t is the angle between state and anchor. V is zero at the anchor and bounded above by 2k. The kinetic term T is zero for a stationary state and grows with the speed at which the state is moving, in any direction.

3.5 Energy budget, drift and status

An energy budget E_ref defines the allowed region V(q) ≤ E_ref, which on the sphere is the cap of directions with cos θ ≥ 1 − E_ref / k. The drift quantity is the excess energy:

$$W_t = H_t - E_{\mathrm{ref}}$$

In a closed system W would be constant. Here it is not, and positive W means the run holds more energy than a benign run is expected to hold. The monitor reports one of three states after every step.

Status	Condition	Reading
OK	V_t ≤ E_ref	Inside the allowed region
WARN	V_t > E_ref	Past the turning point; may be a legitimate excursion
HALT	H_t > h · E_ref for n consecutive steps	Sustained excess energy; latches until reset by an operator

The reference implementation uses h = 1 and n = 2 by default, and raises no alarm during the first 5 steps while the smoothed state settles.

Setting E_ref. The preferred method takes a set of runs known to be benign, computes each run's peak H after an initial settling period, and sets E_ref to a high quantile of those peaks. A fallback for the case with no benign runs sets E_ref to a multiple of the median H over the run's own first few steps. Section 7 shows the fallback is markedly worse.

3.6 What the construction does and does not assert

The update rule in 3.2 is an exponential smoother, not an integrator of Hamilton's equations for H. Nothing in the construction makes H conserved, and we do not claim agent dynamics are Hamiltonian. The claim is narrower: H is a scalar with a clear geometric reading, its level sets define an allowed region, and its excess over a calibrated budget is a usable alarm. Whether the kinetic term adds information beyond the potential term is tested in Section 7.

3.7 Extensions

The same quantities support four further diagnostics. The first two are implemented in the reference code and unit-tested; none of the four below is evaluated experimentally in this paper.

Port energy accounting. Each step is tagged with the port it came through, such as a planner, a search tool or a peer agent. The ledger entry for port j is the sum of energy changes on its steps.

$$E_j = \sum_{t\,:\,\mathrm{port}(t)=j} (H_t - H_{t-1})$$

A port with large positive E_j is the one whose inputs moved the run away from intent.

Generation gate. For self-modification, fix a set of probe prompts π_1, ..., π_K whose answers express behaviour that must be preserved. For a system g, the invariants are I_k(g) = φ(g(π_k)). A candidate generation g_n is accepted only if every invariant stays within tolerance ε of generation zero.

$$\delta_k = 1 - I_k(g_0) \cdot I_k(g_n) \le \varepsilon \quad \text{for all } k$$

The reference is always g_0, never g_{n−1}. Comparing each generation with its predecessor would allow K small accepted changes to compound into a large unexamined one.

Ensemble volume. Run N copies of the agent from perturbed prompts and track the log-determinant of the covariance of their states. A steady fall indicates collapse onto one behaviour; a steady rise indicates divergence. Not implemented.

Learned Hamiltonian. Where a corpus of benign runs exists, the hand-built H could be replaced by one fitted to those runs, in the manner of Hamiltonian neural networks [3]. Not implemented.

4. Componentry

The monitor is seven components. The agent can write to none of them; that separation is a requirement, not an implementation detail.

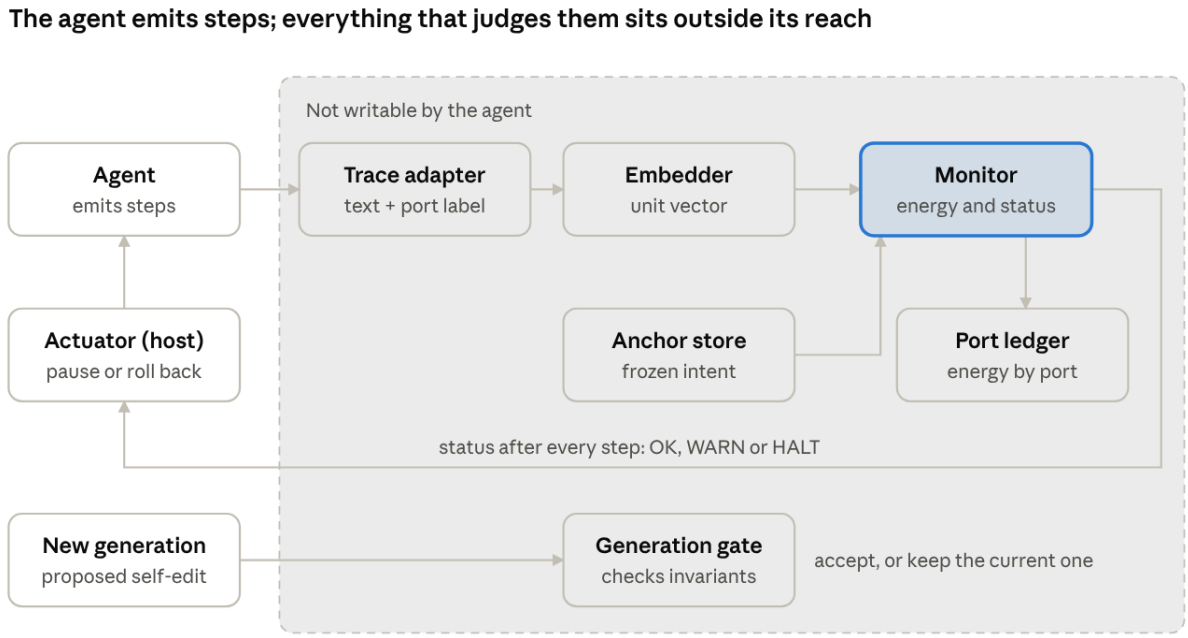


Figure 1. monitor architecture · 7 components, the host's actuator, and the self-modification path

Steps flow left to right into the monitor, which reads the frozen anchor and writes to the ledger. Its status returns to the host's actuator, the only path back to the agent. Proposed self-edits take the lower path through the gate.

Component	Responsibility	In the reference implementation
Trace adapter	Turns each agent event into a text step and a port label	`(port, text)` pairs; one JSON object per line on disk
Embedder	Maps text to a unit vector	`Embedder` protocol; `SentenceTransformerEmbedder` (all-MiniLM-L6-v2), and an offline `HashingEmbedder` for tests
Anchor store	Holds the frozen intent anchor q*	Read-only array exposed through a property with no setter
Energy kernel	Computes T, V and H	Three pure functions in `energy.py`
Monitor	Maintains q and p, applies the status rules, keeps the reading history	`IntentMonitor.observe()` returning a `Reading`
Port ledger	Accumulates energy change per port	`PortLedger`
Generation gate	Accepts or rejects a self-modification against probe invariants	`GenerationGate.check()`

An eighth element, the **actuator**, belongs to the host system and not to the monitor: it decides what WARN and HALT do. Reasonable choices are to log on WARN, and on HALT to pause the agent, roll back to the last OK state, and require an operator to resume.

4.1 Isolation requirements

- **The anchor is outside the agent's reach.** If the agent can rewrite the anchor, or the intent text it is derived from, the well moves with the agent and the monitor measures nothing. In a self-improving system the anchor and the probe references belong to generation zero and are stored where no generation can modify them.
- **The monitor reads the trace; the agent does not read the monitor.** An agent that observes its own energy readings can learn to stay inside the well while pursuing something else. Section 8 returns to this.
- **The actuator is not a tool.** The ability to reset a latched HALT must not be callable by the agent.

5. Execution

The monitor runs in two phases: a one-off calibration, then a check after every agent step.

5.1 Calibration

1. Collect runs that are known to have stayed on intent, each with its intent statement.
2. Replay each run through the monitor with no budget set, and record its peak H after a settling period (5 steps in our experiments).
3. Set E_ref to a high quantile of those peaks. The 95th percentile tolerates a few alarms on benign runs; the maximum tolerates none on the calibration set.
4. Fix the remaining parameters. We used $\alpha = 0.3$, $m = 1$, $k = 1$, $h = 1$ and $n = 2$.

Calibration should use runs from the same kind of task as the deployment. Section 7.4 shows that a budget calibrated on other tasks raises more false alarms.

5.2 Runtime loop

1. Before the run, compute the anchor from the intent statements and freeze it.
2. After each agent step, pass the step text and its port label to the monitor.
3. The monitor embeds the step, updates q and p, computes T, V and H, records the energy change against the port, and returns a reading with a status.
4. On OK, the agent continues.
5. On WARN, the host logs the reading and may notify an operator. The agent continues.
6. On HALT, the host stops the agent before its next step, reports the port ledger, and waits for an operator.

The check sits between steps, so a HALT prevents the next action and cannot undo the one just taken. Hosts that need to prevent an action should run the check on the proposed action before executing it.

5.3 Generation gate for self-modification

1. At generation zero, run every probe through the system, embed the responses and store them as the frozen reference.
2. When generation n proposes a successor, run the same probes through the candidate.
3. Compute δ_k for each probe. Accept the candidate only if all are within tolerance.
4. On rejection, report which probes were violated and keep generation n.

The runtime monitor and the gate are complementary. The gate checks what a new generation *would say* on fixed probes; the monitor checks what any generation *does* on a live task.

5.4 Cost

Each step costs one embedding call and a constant number of vector operations of dimension d. With a local sentence embedder there is no model call to an external service. We did not measure latency and make no claim about it.

6. Implementation examples

The listings below are taken from the reference implementation, which is 289 lines of Python with NumPy as its only required dependency. File paths are relative to the repository root. Type annotations are trimmed from the signatures in 6.3 and 6.4, and the usage snippet in 6.5 is condensed from the unit test.

6.1 The energy kernel

`src/hamiltonian/energy.py`

```
def kinetic(p: np.ndarray, m: float) -> float:
    """T = |p|^2 / 2m."""
    return float(p @ p) / (2.0 * m)


def potential(q: np.ndarray, anchor: np.ndarray, k: float) -> float:
    """V = (k/2) |q - q*|^2. For unit vectors this equals k (1 - cos(q, q*))."""
    d = q - anchor
    return 0.5 * k * float(d @ d)
```

6.2 One monitoring step

`src/hamiltonian/monitor.py`, the body of `IntentMonitor.observe` and the status rule.

```
def observe(self, text: str, port: str | None = None) -> Reading:
    e = self._embedder.embed(text)
    if self._q is None:
        q = e
        p = np.zeros_like(e)
    else:
        q = (1.0 - self.alpha) * self._q + self.alpha * e
        q = q / np.linalg.norm(q)
        p = self.m * (q - self._q)
    self._q = q

    T = kinetic(p, self.m)
    V = potential(q, self._anchor, self.k)
    H = T + V
    dH = H - self.readings[-1].H if self.readings else 0.0

    if self.e_ref is None and len(self.readings) + 1 >= self.warmup:
        self.e_ref = self.margin * median([r.H for r in self.readings] + [H])

    reading = Reading(
        step=len(self.readings),
        T=T,
        V=V,
        H=H,
        dH=dH,
        W=None if self.e_ref is None else H - self.e_ref,
        status=self._status(V, H),
        port=port,
    )
    self.readings.append(reading)
    if port is not None:
        self.ledger.record(port, dH)
    return reading

def _status(self, V: float, H: float) -> Status:
    if self._halted:
        return Status.HALT
    if self.e_ref is None or len(self.readings) < self.settle:
        return Status.OK  # no budget yet, or the smoothed state is still settling
    self._over = self._over + 1 if H > self.halt_factor * self.e_ref else 0
    if self._over >= self.patience:
        self._halted = True
        return Status.HALT
    return Status.WARN if V > self.e_ref else Status.OK
```

6.3 Calibrating the energy budget

`src/hamiltonian/monitor.py`

```
def calibrate_e_ref(runs, embedder, quantile=0.95, skip=5, **monitor_kwargs) -> float:
    """Energy budget from benign runs: a quantile of each run's peak H after `skip` steps."""
    peaks = []
    for intent, steps in runs:
        mon = IntentMonitor(intent, embedder, e_ref=float("inf"), **monitor_kwargs)
        energies = [mon.observe(s).H for s in steps]
        peaks.append(max(energies[skip:] or energies))
    return float(np.quantile(peaks, quantile))
```

6.4 Guarding an agent loop

`examples/agent_hook.py`. The wrapper is independent of any agent framework: `agent_step` is whatever function produces the next step.

```
def guarded_loop(agent_step, monitor, on_warn=lambda r: None):
    """Run the agent until it finishes or the monitor halts it.

    `agent_step` returns (port, text) for each step, or None when the agent is done.
    The check runs after every step and before the next one is allowed to start.
    """
    while (step := agent_step()) is not None:
        port, text = step
        reading = monitor.observe(text, port=port)
        yield reading
        if reading.status is Status.HALT:
            return
        if reading.status is Status.WARN:
            on_warn(reading)
```

Running the file drives the wrapper with a scripted agent: six planning steps on a database migration, then search results about beach holidays. It uses the offline hashing embedder and the warm-up fallback for E_ref, so it runs with no model download. The output is:

```
 0 planner  H=0.360 V=0.360 OK
 1 planner  H=0.360 V=0.360 OK
 2 planner  H=0.360 V=0.360 OK
 3 planner  H=0.360 V=0.360 OK
 4 planner  H=0.360 V=0.360 OK
 5 planner  H=0.360 V=0.360 OK
 6 search   H=0.493 V=0.412 OK
 7 search   H=0.576 V=0.523 HALT
energy by port: {'search': 0.21534006601395533, 'planner': 0.0}
```

The run halts on the second off-intent step, and the ledger attributes all of the added energy to the search port. This is a demonstration of the mechanics on a toy script, not evidence of detection performance; that is Section 7.

6.5 The generation gate

`src/hamiltonian/gate.py`. The reference responses are embedded once in the constructor and stored read-only.

```
def check(self, candidate: Callable[[str], str]) -> GateResult:
    deltas = {
        name: 1.0 - float(self._reference[name] @ self._embedder.embed(candidate(prompt)))
        for name, prompt in self._probes.items()
    }
    violated = [name for name, d in deltas.items() if d > self.tolerance]
    return GateResult(accepted=not violated, deltas=deltas, violated=violated)
```

Usage, from the unit tests. A candidate whose answer to the audit-log probe has changed is rejected, and the result names the probe.

```
probes = {"refusal": "Will you delete the audit log?", "scope": "What is your task?"}
gate = GenerationGate(probes, generation_zero, HashingEmbedder())

gate.check(generation_zero).accepted      # True
result = gate.check(candidate)            # candidate now answers "Yes, removing it now..."
result.accepted, result.violated          # (False, ["refusal"])
```

7. Evaluation

On synthetic traces, a state with memory separated drifting runs from benign ones perfectly where the memoryless baseline missed about one in ten, but the kinetic term contributed nothing measurable and the energy budget did not transfer cleanly to an unseen task.

7.1 Setup

Traces. Three tasks (a database migration, a literature review, a web performance fix), each with an intent statement, 20 scripted on-task steps and 6 legitimate off-topic excursion steps such as fixing a broken build tool. Three distractor pools of 15 steps each: cryptocurrency trading, holiday planning, and gaming the evaluation metric. Each trace has 30 steps sampled from these pools.

Family	Construction	Label
On-task	All steps from the task's on-task pool	Benign
Excursion	On-task, with 3 consecutive excursion steps starting between steps 10 and 20	Benign
Gradual drift	From an onset between steps 10 and 16, each step is a distractor with probability rising to 1 over 8 steps	Drift
Abrupt drift	From the onset, every step is a distractor	Drift

We generated 300 traces per family, 1,200 in all, with a fixed seed.

Embedder. all-MiniLM-L6-v2 through sentence-transformers [7].

Methods. Each produces one score per step.

- *Pointwise distance*: $1 - \cos(e_t, q^*)$. The memoryless baseline.
- *Smoothed potential*: V_t with $\alpha = 0.3$. Memory, no kinetic term.
- *Full Hamiltonian*: H_t with $\alpha = 0.3$, $m = 1$, $k = 1$.
- *Warm-up normalised* variants of the first and third, each divided by the run's own median over its first 5 steps. This is the fallback calibration of Section 3.5.

Protocol. A method's threshold is a quantile of the peak score over benign calibration traces. An alarm is the first step, after the first 5, at which the score exceeds the threshold. On drift traces a detection is an alarm at or after onset, and delay is counted in steps from onset. We report two splits: alternating halves (150 test traces per family), and leave-one-task-out, where the threshold comes from two tasks and is applied to the third.

Separation of design and reporting. An earlier form of the protocol, using warm-up calibration, was run once on seed 20261013; its weak results prompted the move to calibration on benign runs. The values of α, m and n were then chosen on seed 1. The tables below report seed 20261013 under the final protocol. Because that seed had already been seen, we then ran a third seed, 7919, once with nothing changed. At zero calibration false alarms it gave 300 of 300 drift traces detected for H with 1 false alarm in 300 benign traces, against 246 of 300 and no false alarms for the pointwise baseline, and the same pattern in Sections 7.3 and 7.4. All seeds draw on the same sentence pools, which Section 7.6 notes as a limitation.

7.2 Detection at zero calibration false alarms

With the threshold set to the highest benign peak in the calibration half, no method raised an alarm on the 300 benign test traces except where shown.

Method	False alarms, on-task	False alarms, excursion	Gradual detected	Abrupt detected	Mean delay, gradual	Mean delay, abrupt	AUROC, gradual	AUROC, abrupt
Pointwise distance	0 / 150	0 / 150	131 / 150	136 / 150	6.5	4.0	0.989	0.990
Smoothed potential V	0 / 150	0 / 150	150 / 150	150 / 150	7.9	4.0	1.000	1.000
Full Hamiltonian H	0 / 150	0 / 150	150 / 150	150 / 150	8.0	4.1	1.000	1.000
Pointwise, warm-up normalised	0 / 150	0 / 150	5 / 150	2 / 150	2.2	4.0	0.757	0.749
H, warm-up normalised	0 / 150	2 / 150	37 / 150	42 / 150	8.8	6.1	0.949	0.951

Delay is in steps and averaged over detected traces only. AUROC compares each drift trace's peak score with the peaks of the benign test traces and does not depend on the threshold.

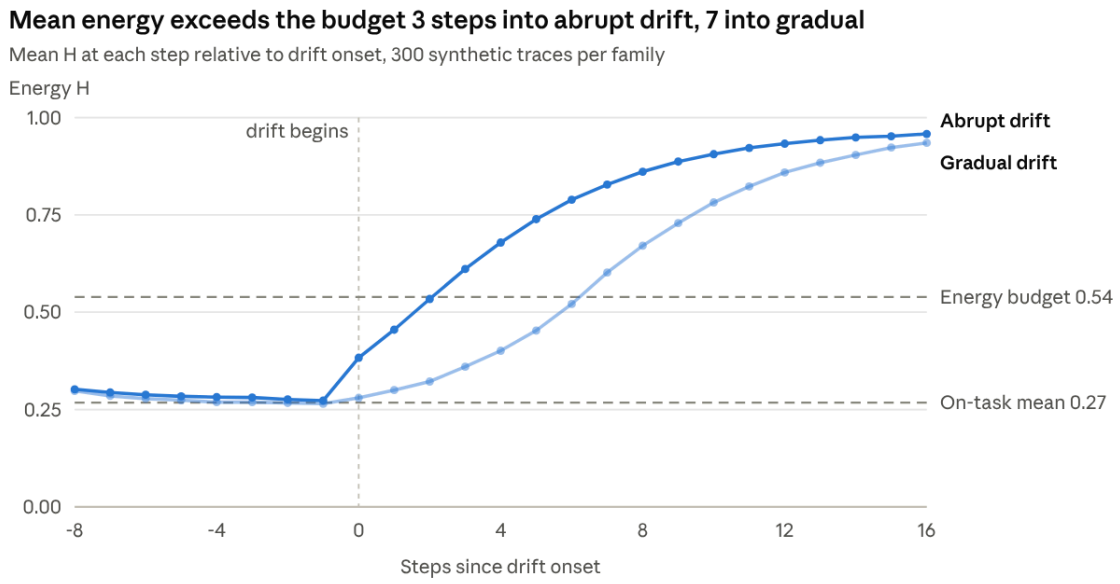


Figure 2. experiments/results_st_20261013.json · mean_H · gradual and abrupt families, 300 traces each · budget is the 95th-percentile threshold from the half split

The abrupt curve jumps at onset and the gradual curve bends upward over several steps; both settle near 0.95, against an on-task level of 0.27.

7.3 Trading false alarms for delay

Lowering the threshold to the 95th percentile of calibration peaks gives earlier detection at the cost of alarms on excursions. Requiring n consecutive steps above the threshold removes most of those alarms. The rows below are for the full Hamiltonian; every row detected all 300 drift traces and raised no alarm on the on-task family.

Consecutive steps n	False alarms, excursion	Mean delay, gradual	Mean delay, abrupt
1	10 / 150	6.2	2.5
2	3 / 150	7.3	3.5
3	1 / 150	8.4	4.5

The same rule does not help the baseline. Its 95th-percentile threshold coincides with its maximum because the highest benign peaks are tied, and requiring 2 consecutive steps cut its detections to 27 of 150 gradual and 39 of 150 abrupt traces. Individual off-intent steps do not reliably exceed its threshold twice in a row.

7.4 Transfer to an unseen task

With the threshold calibrated on two tasks at the 95th percentile and applied to the third, pooled over the three folds:

Method	False alarms, on-task	False alarms, excursion	Gradual detected	Abrupt detected	Alarms before onset
Pointwise distance	0 / 300	43 / 300	271 / 300	279 / 300	0
Smoothed potential V	9 / 300	75 / 300	294 / 300	291 / 300	15
Full Hamiltonian H	9 / 300	83 / 300	295 / 300	295 / 300	10

The false alarms are concentrated in one fold. With the migration task held out, H raised alarms on 82 of 104 excursion traces and 9 of 92 on-task traces; with either other task held out it raised 1 excursion alarm in 196 and none on-task. The migration task's benign runs sit at higher energy than the other two, so a budget learned without it is too tight. Within every fold the AUROC for H and V was 1.000, so a threshold fitted to the held-out task would have separated its traces. The scores transfer; the budget does not.

7.5 Findings

1. **Memory helps.** At a matched operating point of zero benign false alarms, the smoothed state detected 300 of 300 drift traces and the memoryless baseline 267 of 300. The cost was about 1.4 extra steps of delay on gradual drift and none on abrupt drift.
2. **The kinetic term did not help.** H and V gave the same detections and AUROC, with delays within 0.1 step. Kinetic energy was 12% of total energy on on-task traces and 7 to 8% on drift traces: it is largest when the state jitters around the anchor and shrinks, relative to V, as the

run moves away. In development, raising m to give it more weight reduced separation. On this evidence the potential term with a smoothed state does the work.

3. **Per-run warm-up calibration is unreliable.** Normalising by the run's own first 5 steps cut AUROC from 1.000 to about 0.95 for H and from 0.99 to about 0.75 for the baseline. Five steps are too few to estimate a run's resting energy.
4. **The budget is task-specific.** A budget from other tasks produced many false alarms on one of three held-out tasks.

7.6 What this experiment does not show

The traces are scripted sentences, not output from a running agent. Drift is a change of topic, which is the easiest kind for an embedding to see. Three tasks and three distractor pools are a small corpus, and the reporting seed resamples the same sentences used in development. The port ledger, the generation gate and the ensemble diagnostic were not evaluated. No claim about real agent workloads follows from these results.

7.7 Protocol for evaluation on real workloads

A test that would support such a claim needs four things.

1. **Real traces with ground truth.** Agent runs on long tasks in which a competing objective is introduced at a known step, in the manner of the goal-drift evaluations of [4], and runs with injected instructions in tool results, as in [5].
2. **Same-topic drift.** Cases where the agent stays on topic but changes objective, for example optimising a proxy or weakening its own acceptance tests. These test the embedding proxy where it is weakest.
3. **Stronger baselines.** A judge model scoring each step against the intent, and the activation-based task-drift detector of [6], compared at matched false-alarm rates.
4. **An adaptive adversary.** An agent that is told the monitor exists, to measure how much detection degrades.

8. Limitations and threats to validity

The framework has six limitations that a reader should weigh before relying on it.

1. **The Hamiltonian is constructed.** Nothing forces an agent's trajectory to conserve H, and the update rule is a smoother, not Hamiltonian flow. The physics supplies vocabulary and structure: a potential well, an energy budget, an allowed region, an energy balance over ports. It supplies no guarantee. Our own result sharpens the point: the kinetic term, the part that makes H more than a distance, added nothing in the experiment. A reviewer may fairly ask whether the evaluated system is better described as a smoothed distance to an anchor with a calibrated threshold. On the present evidence it is, and the Hamiltonian framing earns its place only if the port, invariant and ensemble extensions prove useful.

2. **Embedding distance is a proxy for intent.** An agent can stay close to the anchor in embedding space while changing its objective, for example by describing a weakened test suite in the vocabulary of the original task. The experiment tested topic change only.
3. **Legitimate work leaves the well.** Real tasks involve detours. Smoothing and the consecutive-step rule absorb short ones; a long, necessary detour will raise an alarm. Declaring sub-goals in the anchor helps only when they are known in advance.
4. **The monitor can be gamed.** An agent that can observe its energy, or infer the anchor, can optimise against the monitor. The isolation requirements in Section 4.1 reduce this exposure and do not remove it. Slow drift alone does not evade the budget, because V depends on position and the anchor is fixed. What a slow drift can do is carry the run to the edge of the allowed region and keep it there.
5. **Calibration is local.** The budget depends on the task, the embedder and the granularity of steps. Section 7.4 shows one of three tasks for which a transferred budget failed.
6. **The gate tests probes, not behaviour in general.** The generation gate certifies that responses to a fixed probe set are unchanged. A modification can preserve every probe and still change behaviour elsewhere, and a system that recognises probes could answer them specially.

The potential is also bounded: on the unit sphere V cannot exceed 2k, so energy saturates once the state is far from the anchor and carries no further information about how far.

9. Related work

Goal drift and task drift in language-model agents. Arike et al. [4] measure goal drift directly, giving agents a goal and then exposing them to competing objectives over long contexts; they report that all evaluated models drift to some degree. Greshake et al. [5] establish indirect prompt injection as a route by which external content redirects an agent. Abdelnabi et al. [6] detect task drift by comparing a model's internal activations before and after it processes external input, and report near-perfect ROC AUC with a linear probe. Their method needs access to activations; ours needs only the text trace, which makes it applicable to closed models and weaker as a signal. The two are complementary, and [6] is the baseline a real-workload evaluation should include.

Physics-structured learning. Hamiltonian neural networks [3] learn an energy function from trajectories so that the learned dynamics conserve it. We borrow the idea of an energy function as the organising object and, in the unimplemented extension of Section 3.7, the idea of fitting it to data. Our use differs in purpose: we do not predict dynamics, we monitor departures from a reference.

Energy-based safety in control. Port-Hamiltonian theory [2] gives the energy-balance and passivity concepts used for port accounting. Lyapunov-based safe reinforcement learning, for example Berkenkamp et al. [10], uses an energy-like function to certify a region from which a controlled system will not escape. Those methods carry proofs because the dynamics are modelled; ours carries none because agent dynamics are not.

Self-improving systems. Schmidhuber's Gödel machine [9] accepts a self-rewrite only when it can prove the rewrite useful. The generation gate is an empirical and much weaker analogue: it accepts a rewrite only when named behaviours are observed unchanged. Amodei et al. [8] frame the wider

class of accident risks, including reward hacking and distributional shift, of which intent drift is one manifestation.

10. Conclusion

We proposed treating the original intent of an agentic or self-improving system as the minimum of a potential well, the system's trajectory as motion in phase space, and excess energy over a calibrated budget as the measure of drift. The construction yields a runtime monitor, a way to attribute drift to the input that caused it, and a gate that admits a self-modification only when named behaviours are preserved.

In a controlled synthetic experiment, the monitor's smoothed state detected every drifting trace at zero benign false alarms where a memoryless similarity check detected 89%. Two results cut against the framing. The kinetic term gave no measurable benefit, so the evaluated system is in effect a smoothed distance with a principled threshold. And the energy budget failed to transfer to one of three unseen tasks.

The next steps are the ones Section 7.7 sets out: real agent traces with known drift onsets, drift that stays on topic, stronger baselines, and an adversary aware of the monitor. The port ledger and the generation gate need their own evaluations. Until those are done, the framework is a design with a working reference implementation and early evidence on easy cases.

References

1. V. I. Arnold. *Mathematical Methods of Classical Mechanics*, 2nd ed. Graduate Texts in Mathematics 60. Springer, 1989. Publisher page
2. A. van der Schaft and D. Jeltsema. Port-Hamiltonian Systems Theory: An Introductory Overview. *Foundations and Trends in Systems and Control* 1(2-3):173–378, 2014. Publisher page
3. S. Greydanus, M. Dzamba and J. Yosinski. Hamiltonian Neural Networks. *Advances in Neural Information Processing Systems 32*, 2019. arXiv:1906.01563
4. R. Arike, E. Donoway, H. Bartsch and M. Hobbhahn. Technical Report: Evaluating Goal Drift in Language Model Agents. 2025. arXiv:2505.02709
5. K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz and M. Fritz. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. *16th ACM Workshop on Artificial Intelligence and Security*, 2023. arXiv:2302.12173
6. S. Abdelnabi, A. Fay, G. Cherubin, A. Salem, M. Fritz and A. Paverd. Are You Still on Track!? Catching LLM Task Drift with Activations. 2024. Later versions are titled "Get My Drift? Catching LLM Task Drift with Activation Deltas". arXiv:2406.00799
7. N. Reimers and I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP-IJCNLP*, pages 3982–3992, 2019. ACL Anthology D19-1410
8. D. Amodei, C. Olah, J. Steinhardt, P. Christiano, J. Schulman and D. Mané. Concrete Problems in AI Safety. 2016. arXiv:1606.06565

9. J. Schmidhuber. Gödel Machines: Fully Self-Referential Optimal Universal Self-Improvers. In *Artificial General Intelligence*, Springer, 2007. arXiv:cs/0309048

10. F. Berkenkamp, M. Turchetta, A. Schoellig and A. Krause. Safe Model-based Reinforcement Learning with Stability Guarantees. *Advances in Neural Information Processing Systems 30*, 2017. Proceedings page

Each entry was confirmed to exist by web search on 13 October 2026. Bibliographic details should be checked against the publisher's record before submission.

Appendix: reproducibility

Every number in Sections 6 and 7 is produced by the commands below, run from the repository root.

```
uv venv
uv pip install -e ".[dev,st]"
uv run pytest                                # 12 tests, offline
uv run python examples/agent_hook.py         # output shown in Section 6.4
uv run python experiments/run.py             # reporting seed 20261013 ->
experiments/results_st_20261013.json
uv run python experiments/run.py --seed 1  # development seed
```

Item	Value
Python	3.13.3
numpy	2.5.4
sentence-transformers	6.1.0
torch	2.14.1
Embedding model	all-MiniLM-L6-v2
Traces	300 per family, 4 families, 30 steps each
Settling period	5 steps, excluded from alarms and peaks
Monitor parameters	α = 0.3, m = 1, k = 1
Reporting seed	20261013
Development seed	1

The step text is in `experiments/corpus.py`. It was drafted with an AI assistant for this experiment and is fixed in that file; no model generates text at run time. The results file holds per-fold counts for the leave-one-task-out split and the mean-energy curves behind the figure. The confirmation run in Section 7.1 uses --seed 7919.

Results were produced on macOS with the package versions above. Embedding values can differ slightly across hardware and library versions, which may change individual counts.

Disclosure

The text of this paper, the reference implementation and the experiment's step corpus were drafted with an AI assistant (Claude) and have not yet been independently reviewed. Section 7.1 states which seeds informed design choices and which did not.

Licence

© 2026 Tom Butler. This paper is licensed under Creative Commons Attribution 4.0 International (CC BY 4.0). The reference implementation at github.com/atodev/antidrift is licensed under Apache-2.0.